Presentation

Bot de trading de cryptomonnaies

2023-03-07

Binance API Endpoints

Account Endpoints

GET /api/v3/account

- Description : Récupère les informations du compte.
- Paramètres :
  - `timestamp` (required) : Horodatage UNIX en millisecondes de la demande.
  - `signature` (required) : Signature de la demande.
  - `recvWindow` (optional) : Fenêtre de réception de la demande.
- Exemple d'URL : https://api.binance.com/api/v3/account?timestamp=123456789&signature=xxxxxxxxxxxxxxx&recvWindow=5000

POST /api/v3/order

- Description : Place une nouvelle commande.
- Paramètres :
  - `symbol` (required) : Paire de trading (ex. BTCUSDT).
  - `side` (required) : Côté de la commande (BUY ou SELL).
  - `type` (required) : Type de commande (LIMIT, MARKET, etc.).
  - `quantity` (required) : Quantité de la commande.
  - `price` (required for LIMIT and STOP_LOSS_LIMIT orders) : Prix de la commande.
  - `stopPrice` (required for STOP_LOSS and STOP_LOSS_LIMIT orders) : Prix de stop de la commande.
  - `timestamp` (required) : Horodatage UNIX en millisecondes de la demande.
  - `signature` (required) : Signature de la demande.
  - `recvWindow` (optional) : Fenêtre de réception de la demande.
- Exemple d'URL : https://api.binance.com/api/v3/order?symbol=BTCUSDT&side=BUY&type=LIMIT&quantity=1&price=10000&timestamp=123456789&signature=xxxxxxxxxxxxxx&recvWindow=5000

Market Data Endpoints

GET /api/v3/ticker/price

- Description : récupérer les prix des paires de trading en temps réel. * Paramétre : Aucun paramétre requis
- Exemple d'URL : https://api.binance.com/api/v3/ticker/price

GET /api/v3/klines

- Description :récupérer les données historiques de prix pour une paire de trading spécifique
- Paramétres :

  - `symbol` : (requis) La paire de trading pour laquelle vous voulez récupérer les données, par exemple "BTCUSDT".
  - `interval` : (requis) L'intervalle de temps pour chaque bougie (ou chandelier) de prix, par exemple "1h" pour une bougie de 1 heure ou "1d" pour une bougie de 1 jour.
  - `startTime` : (optionnel) La date et l'heure de début (en millisecondes) pour récupérer les données historiques. Si non spécifié, la requête commencera par le début des données disponibles pour cette paire de trading.
  - `endTime` : (optionnel) La date et l'heure de fin (en millisecondes) pour récupérer les données historiques. Si non spécifié, la requête récupérera les données les plus récentes disponibles.
  - `limit` : (optionnel) Le nombre de bougies de prix à récupérer. La valeur par défaut est de 500, mais peut être augmentée jusqu'à 1000.
- `Exemple d'url` : https://api.binance.com/api/v3/klines?symbol=BTCUSDT&interval=1h&startTime=1615142400000&endTime=1615826940000&limit=500

Get /api/v3/depth

- Description : récupérer les informations sur le carnet d'ordres d'une paire de trading spécifique, y compris les prix actuels d'achat et de vente et la profondeur de chaque niveau de prix
- Paramètres :
  - `symbol` : (requis) La paire de trading pour laquelle vous voulez récupérer les informations du carnet d'ordres, par exemple "BTCUSDT".
  - `limit` : (optionnel) Le nombre de niveaux de prix à récupérer de chaque côté du carnet d'ordres. La valeur par défaut est de 100, mais peut être augmentée jusqu'à 500.
- `Exemple d'URL`:https://api.binance.com/api/v3/depth?symbol=BTCUSDT&limit=100

GET /api/v3/ticker/24hr

- Description : Récupère les statistiques de 24 heures pour un symbole.
- Paramètres :
  - `symbol` (optional) : Paire de trading (ex. BTCUSDT).
- Exemple d'URL : https://api.binance.com/api/v3/ticker/24hr?symbol=BTCUSDT

GET /api/v3/trades

- Description : récupérer les informations sur les derniers trades effectués pour une paire de trading spécifique
- Paramètres :
  - `symbol` : (requis) La paire de trading pour laquelle vous voulez récupérer les derniers trades, par exemple "BTCUSDT".
  - `limit` : (optionnel) Le nombre de trades à récupérer. La valeur par défaut est de 500, mais peut être augmentée jusqu'à 1000.
- Exemple d'url :https://api.binance.com/api/v3/trades?symbol=BTCUSDT&limit=500

GET /api/v3/exchangeInfo

- Description : récupérer des informations sur les paires de trading disponibles sur la plateforme, y compris les symboles de trading, les limites de prix, les tailles de lots, les types d'ordres, les filtres et autres détails pertinents pour le trading
- Paramétres : Aucun
- Exemple d'url : https://api.binance.com/api/v3/exchangeInfo

Fonctions Utilisées :

```
@staticmethod
def get_ticker_historical_prices_on_binance(client: Client,
                                            ticker: str,
                                            interval: str,
                                            limit: int = 100) -> pd.DataFrame:
    crypto_history = client.klines(ticker, interval, limit=limit)
    columns = ['time', 'open', 'high', 'low', 'close', 'volume', 'close_time', 'quote_asset_volume',
               'nb_of_shares', 'taker_buy_base_asset_volume', 'taker_buy_quote_asset_volume', 'ignore']
    crypto_history_df = pd.DataFrame(crypto_history, columns=columns)
    crypto_history_df['time'] = pd.to_datetime(crypto_history_df['time'], unit='ms')
    return crypto_history_df
```

Cette methode nous donne les prix historiques d'une crypto-monnaie sur Binance. Les paramètres sont les suivants :

- client: une instance de l'API client de Binance.
- ticker: le symbole de la crypto-monnaie, par exemple "BTCUSDT" pour Bitcoin en dollars US.
- interval: l'intervalle de temps pour lequel on veut récupérer les prix historiques, par exemple "1h" pour une heure.
- limit: le nombre de lignes de prix historiques que l'on souhaite récupérer.

La méthode utilise l'API client de Binance pour récupérer les prix historiques de la crypto-monnaie, sous forme de bougies (ou "klines"). Ensuite, elle transforme les données reçues en un objet pandas DataFrame avec des colonnes pour chaque prix et volume de transaction. Elle renvoie le DataFrame ainsi créé.

```
@staticmethod
    def get_recent_trades(client: Client, ticker: str, limit: int=10) -> pd.DataFrame:
        # isBuyerMaker = True => sell transaction, isBuyerMaker = False => buy transaction
        trades = client.trades(ticker, limit=limit)
        trades_df = pd.DataFrame(trades)
        trades_df['time'] = pd.to_datetime(trades_df['time'], unit='ms')
        return trades_df
```

La méthode **get_recent_trades** est une méthode statique qui prend en entrée un objet **Client** de la bibliothèque **binance** ainsi que le nom d'un ticker et un nombre limite de transactions à récupérer.

Cette méthode utilise la méthode **trades** de l'objet Client pour récupérer les dernières transactions pour le ticker donné et la limite spécifiée. Elle crée ensuite un DataFrame pandas à partir de ces transactions.

La méthode convertit également les valeurs de temps en format lisible en utilisant la méthode pd.to_datetime de pandas. Enfin, elle retourne le DataFrame contenant les informations sur les transactions.

```
    @staticmethod
    def get_orderbook(client: Client, ticker: str, limit: int=10) -> pd.DataFrame:
        """ bid/ask """
        depth = client.depth(ticker, limit=limit)
        depth_df = pd.DataFrame(depth)
        return depth_df
```

La méthode envoie une requête à l'API Binance pour obtenir les données du carnet d'ordres pour la paire de devises spécifiée, avec une profondeur de limit. Les données sont renvoyées sous forme d'un dictionnaire contenant deux clés: **bids** pour les ordres d'achat et **asks** pour les ordres de vente, avec des listes de listes contenant le prix et la quantité de chaque commande.

La méthode convertit ensuite les données du carnet d'ordres en un objet DataFrame Pandas, où chaque ligne contient le prix et la quantité d'un ordre d'achat ou de vente, et renvoie cet objet DataFrame.